Blastoids Pic Breeder

An Interactive Evolutionary Selection System Over a Synthetically Generated Image Corpus

Design Notes and Technical Documentation

2025

Abstract

Blastoids Pic Breeder is a browser-based interactive system for shaping a preference field over a corpus of 5,000 synthetically generated images. The user rates images through a CRT-aesthetic targeting interface, progressively building a semantic preference vector in CLIP embedding space. A two-pool sampler uses this vector to bias the presentation stream, while a breeding layer generates mutant offspring from the top-rated set. Offspring carry interpolated latent vectors from their parents and are subject to a generation-indexed decay function, ensuring that the population remains adaptive rather than archival. This document describes the full system: the image generation workflow, the embedding architecture, the preference field dynamics, the breeding operators, and the temporal mechanics that give the system its developmental arc.

Contents

1 Introduction and Motivation **2**

2 Corpus Generation **2**
2.1 Overview . . . 2
2.2 ComfyUI Workflow . . . 3
2.2.1 Model and Sampler Parameters . . . 3
2.2.2 Positive Prompt . . . 3
2.2.3 Negative Prompt . . . 4
2.3 Corpus Statistics . . . 4

3 CLIP Embedding Architecture **5**
3.1 Sidecar Model . . . 5
3.2 Runtime Loading . . . 5
3.3 Why CLIP Changes the System . . . 6

4 Preference Field Dynamics **6**
4.1 The Preference Vector . . . 6
4.2 Item Scoring . . . 7
4.3 Temperature and Sampling . . . 7
4.4 Two-Pool Sampling . . . 7
4.5 Field Concept Readout . . . 8

5 Breeding Layer **8**
5.1 Overview . . . 8
5.2 Mutation Operators . . . 8
5.2.1 Soft Crop . . . 8
5.2.2 Blend . . . 9
5.2.3 Color Shift . . . 9
5.3 Mutation Probability Drift . . . 9
5.4 Generation Decay . . . 10
5.5 Pool Pruning . . . 10
5.6 Offspring Thumbnails and Lineage Ghosting . . . 10

6 Temporal Mechanics **11**
6.1 The Three Clocks . . . 11
6.2 The Flow Field . . . 11
6.3 Image Crossfade . . . 12

7 Interface and Controls **12**
7.1 Modes . . . 12
7.2 Keyboard Controls . . . 12
7.3 HUD Readouts . . . 12

8 Tuning Parameters **13**

9 Persistence **13**

10 Conclusion **13**

1. Introduction and Motivation

Most image curation interfaces are fundamentally sorters. They rank, filter, and retrieve, but they do not evolve. The user's role is to identify what already exists; the system does not use that identification to generate anything new. The result is a one-directional relationship between user and corpus that exhausts itself as soon as the corpus is fully ranked.

Blastoids Pic Breeder is designed around a different premise: that rating is not the end of a process but the input to one. Every classification shapes a preference field Φ, which in turn reshapes what the system presents, how it samples, and what it is capable of breeding. The user is not curating a static collection but sculpting a distribution over image space. Over time, that sculpted distribution generates descendants.

The system has three structural layers that are intended to be mutually reinforcing rather than merely coexistent.

The first is a *targeting interface* — a green monochrome CRT display framed as a scanning instrument rather than a gallery. This framing is intentional: it reinterprets the act of rating as signal detection rather than passive preference. The user acquires targets; the field responds.

The second is a *preference field* anchored in CLIP embedding space. Unlike systems that use pseudo-random latent vectors, the preference vector Φ in Blastoids corresponds to a direction in a space trained on image-text pairs. Marking architectural photography as AWESOME does not just increase the probability of seeing more of those specific images — it increases the probability of seeing anything that shares their semantic structure.

The third is a *breeding layer* that generates new images by compositing, cropping, and blending parents from the top-rated set. Bred targets carry latent vectors interpolated from their parents and compete in the same preference field as base-corpus images. They are subject to decay and pruning, creating an ecosystem with turnover rather than accumulation.

2. Corpus Generation

2.1. Overview

The base corpus consists of 5,000 images at 256×256 pixels, generated using Stable Diffusion v1.5 via a ComfyUI workflow. The images are named `blastoids_00001_.png` through `blastoids_05000_.png` with five-digit zero-padded indices and a trailing underscore before the extension. They are split across two directories (`images/a/` for indices 1–2500, `images/b/` for 2501–5000) to accommodate repository size constraints, with a corresponding 140×140 thumbnail mirror under `images/thumbnails/`.

2.2. ComfyUI Workflow

The generation pipeline is a standard ComfyUI graph with the following node structure:

```
Node topology:
CheckpointLoaderSimple (id:4) → KSampler (id:3)
EmptyLatentImage (id:5) → KSampler (id:3)
CLIPTextEncode positive (id:6) → KSampler (id:3)
CLIPTextEncode negative (id:7) → KSampler (id:3)
KSampler (id:3) → VAEDecode (id:8) → SaveImage (id:9)
```

2.2.1. Model and Sampler Parameters

Parameter	Value
Checkpoint	v1-5-pruned-emaonly.ckpt
Image dimensions	512×512 (latent), decoded to 512×512
Batch size	1
Steps	20
CFG scale	8
Sampler	Euler
Scheduler	Normal
Denoise	1.0
Seed	Randomized per image
Output prefix	blastoids

Table 1: KSampler and generation parameters from the ComfyUI workflow.

2.2.2. Positive Prompt

The positive conditioning text establishes the visual language of the corpus — an intersection of retro arcade aesthetics, abstract geometry, and semantic field visualization:

Listing 1: Positive CLIPTextEncode prompt (node id:6)

```
retro 3D bubble shooter game scene, floating glowing spheres arranged
in layered depth, player viewpoint aiming reticle at center, spheres
labeled with abstract symbols, minimalist vector wireframe environment,
black background, neon green phosphor glow, CRT monitor aesthetic,
scanlines, slight bloom, high contrast, geometric clarity, early 1990s
arcade style, desaturated except green, subtle perspective grid, smooth
spherical shading, no textures, no clutter, precise composition,
```

```
center-focused gameplay view, sharp lines, low noise

each sphere represents a semantic node, varying size and radius, slight
motion trails indicating trajectories, field-like arrangement instead of
fixed rows, emergent clustering, minimal HUD, abstract mathematical
symbols on spheres, clean spatial separation, no cartoon elements, no
characters, no background scenery

asteroids, space, game, 4x, 3d, wireframe asteroids drifting through
neon-green vector field, swirling flows around them bending light,
minimalistic but high detail, crisp CRT glow, dynamic and energetic

glowing green wireframe, retro terminal aesthetic, force-field
appearance, intricate symmetry
```

The prompt encodes a coherent visual identity that aligns with the application's CRT interface: phosphor green on black, scanlines, wireframe geometry, abstract symbols. This coherence is deliberate — it means that CLIP embeddings of the corpus will cluster around a set of related visual concepts rather than distributing uniformly across the full manifold of natural images.

2.2.3. Negative Prompt

The negative conditioning suppresses features that would break the aesthetic or introduce visual noise:

Listing 2: Negative CLIPTextEncode prompt (node id:7)

```
cartoon bubbles, bright multicolor palette, characters, fantasy
environment, UI clutter, text overlays, logos, realistic textures,
photorealism, explosions, weapons, complex backgrounds, text,
watermark, ugly, amateur, distorted, garish, gawdy, abstract
```

2.3. Corpus Statistics

At 5,000 images with randomized seeds, the corpus samples broadly from the conditional distribution defined by the positive and negative prompts. Because the prompt is multipart and contains some degree of semantic tension (e.g., "asteroids,

space, game" alongside "no cartoon elements, no characters"), the resulting distribution is internally varied while remaining coherent as a whole. Images cluster around several recognizable themes: field visualizations with wireframe bubbles, abstract symbol arrays, targeting reticles, and topographic flow patterns.

This internal variety is precisely what makes the preference field meaningful. If all images were visually identical, Φ would carry no discriminative information. Because they vary within a constrained manifold, the preference field can develop genuine directionality.

3. CLIP Embedding Architecture

3.1. Sidecar Model

Rather than loading a vision model in the browser, the system uses a precomputed embedding sidecar. The Python script `generate_embeddings.py` walks the corpus in index order (matching `IMAGE_URLS[0]` through `IMAGE_URLS[4999]` exactly), embeds each image using OpenCLIP ViT-B/32, and writes a JSON file:

Listing 3: Sidecar JSON structure

```
{
  "model":  "ViT-B-32/openai",
  "dim":    512,
  "count":  5000,
  "embeddings": {
    "0": [0.023, -0.041, ...],  # L2-normalized, 512 floats
    "1": [...],
    ...
  }
}
```

At 512 dimensions with 5,000 images, the uncompressed sidecar is approximately 10 MB. An optional PCA reduction flag produces a smaller file; 64 dimensions retains roughly 85% of explained variance for photographic content and reduces the file to ≈1.3 MB.

3.2. Runtime Loading

The sidecar is fetched asynchronously at startup. The loading sequence is:

1. Hydrate persisted session state from `localStorage` using whatever dimensionality was previously saved.
2. Fetch and parse `blastoids_embeddings.json`.
3. If the loaded dimension differs from the saved dimension (e.g., a pseudo-random 16d session is upgraded to CLIP 512d), clear all offspring and reset the breeding generation counter. Base-image ratings are preserved.
4. Replace each base item's `latent` field in-place with the corresponding CLIP

vector.

5. Resize `preferenceVector` to the new dimension and fill with zeros.
6. If there are existing ratings in the session, call `rebuildPreferenceVector()` to reconstruct Φ in the new embedding space.

If the sidecar is absent or fails to fetch, the system falls back silently to pseudo-random 16-dimensional vectors. Everything still functions; only the semantic coherence of Φ is affected.

3.3. Why CLIP Changes the System

In pseudo-latent mode, the dot product $\Phi \cdot v_i$ for item i is arbitrary: items that look similar may have very different vectors. Marking a group of images as AWESOME builds a preference vector that points in a random direction in a random space. The sampler responds, but only in the sense that it memorizes proximity to previously rated items rather than generalizing to visually similar ones.

In CLIP mode, the dot product $\Phi \cdot v_i$ measures cosine similarity in a space where nearby vectors correspond to semantically similar images. Marking architectural photography as AWESOME pushes Φ toward a direction that genuinely aligns with other architectural photographs, even ones the user has never seen. The preference field becomes a genuine generalization of expressed taste rather than an interpolation of encountered items.

4. Preference Field Dynamics

4.1. The Preference Vector

The preference vector $\Phi \in \mathbb{R}^d$ (where $d = 512$ with CLIP, or $d = 16$ in fallback mode) is the central state object of the system. It is initialized to zero and updated by a weighted sum of rated item latents:

$$\Phi = \frac{\tilde{\Phi}}{\|\tilde{\Phi}\|}, \quad \tilde{\Phi} = \sum_{i \in \text{rated}} w_i \cdot v_i \tag{1}$$

where the weight w_i is positive for favored items and negative for rejected ones:

$$w_i = \text{sgn}(r_i) \cdot (1.5 + 0.12|r_i|) \tag{2}$$

The rating scalars are: AWESOME $= 1.0$, OK $= 0.4$, HORRIBLE $= -1.0$. OK is set above zero rather than at zero because a neutral rating still carries information about the field — it establishes that the item was seen and not strongly rejected, which matters for diversity maintenance. Zero-weighted items would contribute nothing to Φ and would effectively be treated as unseen.

4.2. Item Scoring

Each item receives a composite score combining preference alignment, explicit rating, novelty, and penalties:

$$s_i = \alpha \cdot (\Phi \cdot v_i) + \beta \cdot r_i + \gamma_{\text{novelty}}(n_i) - \delta_{\text{penalty}}(n_i) + \epsilon_{\text{offspring}}(i) - \tau_{\text{age}}(i) \tag{3}$$

The terms are:

Term	Expression	Notes
Alignment	$\alpha = 0.55$	Reduced from 1.0 to prevent CLIP field lock
Explicit rating	$\beta = 0.55$	Direct user signal
Novelty	$\max(0,\ 0.22 - 0.025n_i)$	Unseen items get +0.38
Repeat penalty	$\min(0.45,\ 0.02n_i)$	Grows with view count n_i
Offspring bonus	+0.15 if unseen offspring	Encourages exploration
Age penalty	$(g - g_i) \cdot 0.04$	g = current generation

Table 2: Scoring function components. n_i = view count; g_i = birth generation for offspring.

The alignment weight $\alpha = 0.55$ is a critical design parameter. In CLIP space, a dot product between a preference vector built from a few strong ratings and a semantically aligned item can be as high as 0.7–0.9. At $\alpha = 1.0$, this would push alignment above the sum of all other terms after just 2–3 strong ratings, causing the system to lock onto a narrow concept cluster very quickly. At $\alpha = 0.55$, alignment remains competitive with novelty and explicit rating, maintaining exploration pressure even in a strongly converged field.

4.3. Temperature and Sampling

Items are sampled via a softmax distribution over scores with a temperature parameter that decays with session depth:

$$T(n) = T_{\text{min}} + (T_{\text{max}} - T_{\text{min}}) \cdot e^{-n/\tau} \tag{4}$$

where n is the total number of ratings, $T_{\text{max}} = 1.55$, $T_{\text{min}} = 0.42$, and $\tau = 180$. The temperature floor $T_{\text{min}} = 0.42$ is higher than might be used in a non-CLIP system because in CLIP space the alignment signal is strong enough to make lower floors produce near-greedy selection. At $T_{\text{min}} = 0.42$, roughly 15% more probability mass rests on non-top items at full convergence, preserving the "search" character of the slideshow even in mature sessions.

4.4. Two-Pool Sampling

The sampler maintains separate base-corpus and offspring pools, mixing between them at a controlled ratio:

Listing 4: Two-pool sampler

```
function chooseNext(){
  const useOffspring = offspringItems.length > 0
    && Math.random() < OFFSPRING_MIX_RATIO;  // 0.22

  if(useOffspring){
    const result = sampleFromPool(offspringItems);
    if(result) return result;
  }
  return sampleFromPool(baseItems);
}
```

With `OFFSPRING_MIX_RATIO = 0.22`, approximately one in five draws comes from the bred pool when offspring exist. This decouples offspring visibility from scored dominance: a newly bred target with no ratings will still appear in the stream at a stable rate regardless of whether the preference field happens to align with it. The field shapes which offspring appear, not whether they appear at all.

4.5. Field Concept Readout

The HUD exposes a nearest-neighbor probe of the current field direction: the base item whose latent vector maximally aligns with the normalized Φ:

$$i^* = \arg\max_{i \in \text{base}} \frac{\Phi}{\|\Phi\|} \cdot v_i \tag{5}$$

The probe uses a hysteresis margin: the displayed item only changes when a new best exceeds the current best by at least 0.02 cosine units, preventing flicker from small perturbations to Φ. In CLIP space, this readout approximates the visual concept the field is converging toward. The label is reset whenever Φ is rebuilt from scratch, ensuring that phase transitions (e.g., after a strong burst of new ratings) are reflected correctly.

5. Breeding Layer

5.1. Overview

Breeding is triggered manually by the user (keyboard shortcut `B`). Each press increments the breeding generation counter g and generates up to 12 new bred targets from the current top-rated set. Parents are selected in weighted pairs, with selection probability proportional to item score. Three mutation operators are available.

5.2. Mutation Operators

5.2.1. Soft Crop

A region of parent A is extracted and scaled to fill the full frame. The crop is defined by a quadrant flag $(q_x, q_y) \in \{0, 1\}^2$ and two proportional dimensions $(c_w, c_h) \in [0.45, 0.65]^2$ sampled at breed time:

$$s_x = q_x \cdot (W_{\text{img}} - c_w \cdot W_{\text{img}}) \tag{6}$$
$$s_y = q_y \cdot (H_{\text{img}} - c_h \cdot H_{\text{img}}) \tag{7}$$

The crop parameters are encoded into the offspring's URL fragment (`#crop-qx-qy-cwPct-chPct`) at breed time, making the rendered result deterministic across frames and sessions without storing additional state. This operator preserves strong local features from the parent, biasing the population toward "zooming into good regions."

5.2.2. Blend

Parent A is composited with parent B at a random opacity $\alpha \in [0.30, 0.70]$. In CLIP space, the latent vector is a matching interpolation between the two parent latents at the same ratio, so the semantic and visual representations are mutually consistent:

$$v_{\text{child}} = \text{normalize}\left((1-t) \cdot v_A + t \cdot v_B + \eta\right), \quad \eta \sim \mathcal{N}(0,\ 0.05^2) \tag{8}$$

Blend is the semantically deepest operator: it produces children whose embedding genuinely interpolates between two concepts rather than selecting a region of one. The added noise η ensures that repeated blending of the same parents does not collapse to identical offspring.

5.2.3. Color Shift

A screen-mode tint overlay (warm: `rgba(80,25,0,0.14)` or cool: `rgba(0,15,55,0.14)`) is applied over parent A. This is the weakest operator — predominantly cosmetic, with a slightly higher latent noise term ($\sigma = 0.12$) to compensate for the lack of structural change.

5.3. Mutation Probability Drift

Mutation operator probabilities are not fixed; they shift with breeding generation:

$$p_{\text{crop}} = \max(0.20,\ 0.45 - g \cdot 0.015) \tag{9}$$
$$p_{\text{blend}} = \min(0.55,\ 0.35 + g \cdot 0.015) \tag{10}$$
$$p_{\text{shift}} \approx 1 - p_{\text{crop}} - p_{\text{blend}} \tag{11}$$

At generation 0, crop probability is 45% and blend is 35%. At generation 7, they cross over. By generation 10 and beyond, blend is at its ceiling of 55% and crop has retreated to its floor of 20%. This produces a developmental arc: early sessions extract strong features from individual images; later sessions synthesize across concepts. The transition happens without user intervention and is not visible as a discrete event, but perceptible over time.

5.4. Generation Decay

Each offspring carries a `generation` field g_i set at birth. The age penalty in the scoring function is:

$$\tau_{\text{age}}(i) = \max(0,\ (g - g_i) \cdot k_{\text{decay}}), \quad k_{\text{decay}} = 0.04 \tag{12}$$

At $k = 0.04$, an unrated offspring from generation 1 loses approximately 0.04 scoring points per subsequent breed cycle. After 5 generations, its alignment advantage over fresh items has to exceed 0.20 to remain competitive. This creates natural turnover without requiring explicit culling.

Rated offspring are protected: the explicit rating term ($\beta = 0.55$) can override decay for items the user has invested in. An offspring marked AWESOME ($r = 1.0$) with $\beta \cdot r = 0.55$ accumulates approximately $0.55/0.04 \approx 14$ generations of protection before the age penalty exceeds the explicit signal. Items the user ignored simply sink.

5.5. Pool Pruning

When the offspring pool exceeds 85% of its cap (102 of 120 items), the system prunes low-scoring unrated offspring to make room for fresh ones:

Listing 5: Probabilistic pruning on breed

```
const unrated = offspringItems
  .filter(it => it.label === null)
  .sort((a, b) => itemScore(a) - itemScore(b));

const toPrune = Math.min(
  Math.ceil(OFFSPRING_PER_BREED * 0.5),
  unrated.length
);
```

Pruning is sorted by current score, so it preferentially removes items that are both old (decay penalty) and poorly aligned with the current field. Rated items are never pruned regardless of score.

5.6. Offspring Thumbnails and Lineage Ghosting

Offspring thumbnails are rendered into 160×120 offscreen canvases using the same `drawImageItem` and `applyMonochromeRect` functions as the main display, so the side panel shows the actual mutation rather than the unmodified parent image. The rendered data URL is cached on the item after first generation (`item.thumbDataUrl`), with a deduplication guard preventing multiple simultaneous renders for the same item when the panel refreshes before a slow source image loads.

In the curate panel, hovering over a bred target cycles through its parents as a background overlay at 420 ms intervals. For parents that are themselves bred targets, the ghost shows their cached thumbnail rather than their source image, making lineage visually traceable through multiple generations.

6. Temporal Mechanics

6.1. The Three Clocks

One of the more important structural decisions in the system is separating motion and time into three independent subsystems:

Clock	Variable	Role
Aesthetic clock	`tunnelPhase`	Tunnel ring animation, visual continuity
Difficulty field	`depth`	Bubble count target, speed curve
Player metric	Projectile velocity	Constant scale, independent of depth

Table 3: Three independent time subsystems. Previously all three were coupled through `dt`, producing superlinear perceived acceleration.

Previously, speed was defined as:

```
// Old: t^2 behavior --- manageable then suddenly uncontrollable
const speed = BUBBLE_SPEED_BASE + depth * 0.000004;
```

Since both `depth` and `speed` were proportional to `dt`, and `depth` itself accumulated over time, velocity grew roughly as t^2 from the player's perspective. The replacement uses logarithmic scaling:

$$v(d) = v_0 \cdot (1 + \ln(1 + d) \cdot 0.45) \tag{13}$$

This is bounded above and reaches approximately $2.6v_0$ at depth 100, compared to unbounded linear growth in the original formulation.

6.2. The Flow Field

Bubble motion is driven by a shared flow field rather than independent angular velocities:

$$\frac{d\theta_i}{dt} = k \cdot [0.22 \sin(2\theta_i + \omega_1 t) + 0.12 \sin(\omega_2 t)] \tag{14}$$

All bubbles follow the same underlying drift function, creating the visual impression of a coherent medium rather than independent trajectories. A weak attractor term pulls all angles toward a slowly rotating target, creating loose lane structure:

$$\frac{d\theta_i}{dt} \mathrel{+}= \mu \cdot (\theta^*(t) - \theta_i), \quad \theta^*(t) = \sin(\omega_3 t) \cdot \pi \tag{15}$$

The attractor is weak enough that bubbles maintain their individual positions but strong enough that streams form and dissolve visibly over the course of a session.

6.3. Image Crossfade

Each transition between images in the slideshow performs a 380 ms crossfade. The outgoing image decays to zero at a maximum opacity of 0.72 (never fully opaque, always reading as residue), while the incoming image fades in from opacity $1.4-\alpha_{\text{out}}$. The asymmetry is intentional: the past is lighter than the present.

7. Interface and Controls

7.1. Modes

The system operates in two primary modes.

Slideshow mode advances automatically at 2.8-second intervals, showing images drawn from the two-pool sampler. The user rates each image with keys 1/2/3 or on-screen buttons. Each rating updates Φ and advances to the next sample.

Curate mode pauses autoplay. The user can navigate manually, browse the top and worst ranked sets and the bred pool, and click any thumbnail to jump to that item.

7.2. Keyboard Controls

Key	Action
`1`	AWESOME ($r = 1.0$)
`2`	OK ($r = 0.4$)
`3`	HORRIBLE ($r = -1.0$)
$\rightarrow$	Next image
`M`	Toggle slideshow/curate
`B`	Breed from top set
`T`	Open top-ranked set
`W`	Open worst-ranked set
`O`	Open bred targets
`Space`	Pause/resume
`H`	Hide/show UI

Table 4: Keyboard shortcuts.

7.3. HUD Readouts

- **PREFERENCE** Φ: Magnitude of the preference vector, $\|\Phi\|$. Zero at session start; grows as ratings accumulate. Values above ≈ 0.3 indicate a field with directional structure.
- **TEMPERATURE**: Current sampling temperature $T(n)$. Starts at 1.55, decays toward 0.42 over approximately 180 ratings.
- **FIELD** $\rightarrow$: Nearest-neighbor proxy for Φ's direction. The index of the base-corpus image with highest cosine similarity to Φ. Stabilized with a 0.02 hysteresis margin.

- **LINEAGE**: For base images: `BASE CORPUS`. For bred targets: mutation type and parent IDs, e.g., `blend-0.52 ← [143, 897]`.

8. Tuning Parameters

The following constants are exposed at the top of `index.html` and are the primary dials for adjusting system behavior:

Constant	Default	Effect
`ALIGN_WEIGHT`	0.55	Alignment contribution to scoring. Higher = faster convergen
`T_MIN`	0.42	Sampling temperature floor. Higher = more exploration at co
`T_TAU`	180	Ratings to 63% temperature decay. Higher = exploratory for
`OFFSPRING_MIX_RATIO`	0.22	Fraction of draws from bred pool.
`OFFSPRING_DECAY_RATE`	0.04	Score penalty per generation for unrated offspring.
`AUTOPLAY_MS`	2800	Milliseconds per image in slideshow mode.

Table 5: Exposed tuning constants.

9. Persistence

Session state is serialized to `localStorage` under the key `blastoids_state_v1` after every meaningful state change. The serialized state includes ratings (scalar and categorical label), the preference vector, the offspring pool (including URL fragments encoding mutation parameters), the breeding generation counter, mode, and the current item identity.

The categorical label system (`item.label` ∈ `{awesome, ok, horrible, null}`) is stored alongside the scalar rating to decouple bookkeeping from precise float equality. This means future changes to rating scalars (e.g., adjusting the OK value from 0.4 to 0.35) will not corrupt session statistics.

On reload, the system first hydrates state, then loads the CLIP sidecar asynchronously. If the sidecar dimension differs from the saved preference vector dimension, the vector is discarded and rebuilt from saved ratings using the new embeddings. Offspring with mismatched dimensions are cleared. The design principle is that ratings survive across embedding upgrades; synthesized products do not, because they belong to the coordinate system that generated them.

10. Conclusion

Blastoids Pic Breeder is not primarily a viewer or a ranker. It is a preference field shaping interface. The user constructs an aesthetic direction in embedding space through repeated acts of classification; the system uses that direction to reshape what it presents, what it breeds, and what it decays.

The most important design decisions are the ones that prevent collapse: the alignment weight that keeps Φ from locking too fast, the temperature floor that maintains

exploration pressure in converged sessions, the generation decay that prevents old offspring from sedimenting the pool, and the two-pool sampler that keeps bred targets present at a stable rate regardless of their scored dominance.

The system has three layers — targeting interface, preference field, and breeding layer — that are intended to reinforce a single idea: that the act of looking at images is the act of constructing a field, and that field can produce descendants.

Generated corpus: 5,000 images · SD v1.5 · ComfyUI · Euler sampler · 20 steps · CFG 8
Embedding model: CLIP ViT-B/32 (OpenAI) via OpenCLIP
Interface: Single-file HTML/JS/Canvas · localStorage persistence · Mobile touch support